Aula de hoje: porta laser, textos nos botões e placa do dono

29/10/2026 · Instrutor Pedro Leonardo

O que vamos fazer hoje

Hoje a base ganha uma defesa: uma porta laser que só o dono atravessa. Ela é feita de duas partes, e as duas ficam numa pasta própria, do mesmo jeito que os outros itens da base.

- **Alavanca**: ao chegar perto, aparece a opção de apertar **E**. Ela liga e desliga o laser e fica verde (ligado) ou vermelha (desligado).
- **Laser**: com a alavanca ligada, fica visível e mata qualquer um que atravesse, menos o dono da base. Desligado, fica invisível e não machuca ninguém.
- **Efeitos**: partículas e luzes (LED) nos lasers e na alavanca, que ligam, desligam e mudam de cor sozinhos.
- **Textos flutuantes nos botões**: cada botão de compra mostra em cima dele o que vende e quanto custa.
- **Placa do dono**: uma placa que mostra quem é o dono da base. Quando o dono morre, ele renasce na frente dela.

No fim da aula, cada base vai ter uma porta que protege a área do dono.

Lembrando a regra de sempre: as configurações (cores, transparência, distância) ficam em variáveis no topo do script. Quer mudar a porta? Mude só essas linhas.

Etapa 1: organizar as pastas

Por quê: o script da porta trata a pasta como um pacote. Ele procura uma peça chamada `Alavanca` e considera todas as outras peças da pasta como laser. Ele também sobe pelas pastas para descobrir de qual base a porta é, e assim saber quem é o dono.

Como fazer: monte o Explorer assim:

```
Tycoons
└── Tycoon1
    └── Botao4                (pasta de compra, se a porta for vendida)
        ├── Botao
        └── PortaLaser        (pasta)
            ├── PortaLaser    (Script)
            ├── Alavanca      (Part)
            │   ├── ProximityPrompt
            │   └── PointLight (opcional: LED da alavanca)
            ├── Laser1        (Part)
            │   └── ParticleEmitter (opcional: partículas)
            └── Laser2        (Part, quantos quiser)
```

- A pasta `PortaLaser` precisa estar **dentro de uma base**. Pode ficar solta na base (aparece sempre) ou dentro de uma pasta de compra (aparece quando o botão é comprado).
- A alavanca se chama exatamente `Alavanca`, com A maiúsculo.

- Os lasers podem ter qualquer nome. Toda peça da pasta que não for a alavanca vira laser, inclusive peças dentro de Models.
- O script fica **direto na pasta** `PortaLaser`, e não dentro de uma peça.

Etapa 2: criar a alavanca

Por quê: o dono precisa de um jeito de abrir e fechar a porta. O `ProximityPrompt` é o objeto do Roblox que mostra a tecla **E** na tela quando o jogador chega perto.

Como fazer:

1. Clique com o botão direito na pasta onde a porta vai ficar, vá em **Insert Object** e escolha **Folder**. Renomeie para `PortaLaser`.
2. Dentro da `PortaLaser`, insira uma **Part** e renomeie para `Alavanca`.
3. Deixe a peça pequena, por exemplo Size `1, 3, 1`, posicione do lado da porta e marque **Anchored**. A cor não importa: o script pinta a alavanca de verde ou vermelho.
4. Clique com o botão direito na `Alavanca`, vá em **Insert Object** e escolha **ProximityPrompt**. Não precisa mudar nenhuma propriedade, o script configura tudo.

Crie o `ProximityPrompt` no Studio, e não deixe para o script criar. Um prompt criado no meio do jogo não é anotado pelo botão de compra, e o **E** continuaria aparecendo na alavanca escondida depois de um reset da base.

Etapa 3: criar os lasers

Por quê: os lasers são a parte que bloqueia a passagem. Eles não têm colisão (ninguém bate neles), mas percebem quem passa por dentro.

Como fazer:

1. Dentro da `PortaLaser`, insira uma **Part** e dê um nome, por exemplo `Laser1`.
2. Estique a peça para tapar a passagem. Um laser fino fica bom, por exemplo Size `0.3, 8, 0.3`.
3. Use o material **Neon** e uma cor vermelha forte.
4. Aperte **Ctrl+D** para duplicar e espalhe quantos lasers quiser, formando uma grade.
5. Deixe os lasers visíveis no Studio, para enxergar onde estão. O script ajusta a transparência quando o jogo começa.

Etapa 4: o script da porta laser

Como fazer: clique com o botão direito na pasta `PortaLaser`, insira um **Script**, renomeie para `PortaLaser` e cole o script da porta laser entregue em aula.

Não é preciso entender cada linha. Abaixo estão as partes mais importantes do script, com a explicação de cada uma.

Entendendo o código:

Configurações

```
local comecaLigado = false                      -- estado do laser quando o jogo
começa
local apenasDonoUsa = true                      -- só o dono da base usa a
alavanca
local corLigado = Color3.fromRGB(0, 255, 0)     -- cor da alavanca com o laser
ligado
local corDesligado = Color3.fromRGB(255, 0, 0)  -- cor da alavanca com o laser
desligado
local transparenciaLigado = 0.3                 -- transparência do laser ligado
local transparenciaDesligado = 1                -- transparência do laser
desligado
local distanciaInteracao = 8                    -- distância para aparecer a tecla
E
```

- `comecaLigado` decide se a porta já começa fechada quando alguém pega a base.
- `apenasDonoUsa` decide quem pode mexer na alavanca. Com `true`, só o dono.
- `corLigado` e `corDesligado` são as cores da alavanca (e das luzes dela) em cada estado.
- `transparenciaLigado` e `transparenciaDesligado` controlam o quanto o laser aparece. Veja a Etapa 5.
- `distanciaInteracao` é a distância, em studs, para a tecla E aparecer.

Preparação

```
prompt.ObjectText = "Porta laser"
prompt.KeyboardKeyCode = Enum.KeyCode.E
prompt.HoldDuration = 0
prompt.MaxActivationDistance = distanciaInteracao
```

- O `ProximityPrompt` recebe o texto, a tecla E e a distância. `HoldDuration = 0` faz a alavanca funcionar com um toque na tecla, sem segurar.
- O laço do começo separa a pasta em três listas: os **lasers**, os **efeitos dos lasers** e os **efeitos da alavanca**. Os lasers ficam ancorados, sem colisão e com `CanTouch` ligado.

Funções

```
-- mata quem encosta no laser ligado, menos o dono
local function tocarLaser(hit)
    if not ligado then return end

    local personagem = hit.Parent
    local humanoid = personagem and personagem:FindFirstChildOfClass("Humanoid")
    if not humanoid or humanoid.Health <= 0 then return end

    local jogador = Players:GetPlayerFromCharacter(personagem)
    if jogador and ehDono(jogador) then return end

    humanoid.Health = 0
end
```

- `definirEstado` é o coração da porta. Ela muda a cor da alavanca e das luzes dela, muda a transparência dos lasers, liga ou desliga os efeitos dos lasers e troca o texto do E entre "Ligar laser" e "Desligar laser".

- `tocarLaser` roda quando algo encosta num laser. Se o laser está ligado e quem encostou tem um `Humanoid` (é um personagem) e não é o dono, a vida dele vai para 0.
- `ehDono` usa a função do módulo `Economia` para conferir se o jogador é o dono da base.

Eventos

```
prompt.Triggered:Connect(function(jogador)
    if apenasDonoUsa and tycoon and not ehDono(jogador) then return end
    definirEstado(not ligado)
end)
```

- `prompt.Triggered` dispara quando alguém aperta E. Se for o dono, a porta troca de estado: `not ligado` inverte, ligado vira desligado e desligado vira ligado.
- Quando a base troca de dono ou fica livre, a porta volta ao estado de `comecaLigado`.

Etapa 5: deixar o laser mais ou menos transparente

Por quê: um laser totalmente sólido parece uma parede. Um pouco transparente, ele parece um feixe de luz e o jogador vê o que tem do outro lado.

Como fazer: no topo do script, mude a linha:

```
local transparenciaLigado = 0.3   -- transparência do laser ligado (0 = sólido, 1 = invisível)
```

Escolha um número entre 0 e 1:

Valor	Como o laser fica
`0`	Totalmente sólido, parece uma parede
`0.3`	Forte, mas dá para ver um pouco através (padrão)
`0.5`	Meio a meio, bem com cara de feixe de luz
`0.7`	Bem fraco, quase um brilho
`1`	Invisível (o laser continua matando!)

- Com o material **Neon**, o laser continua brilhando mesmo bem transparente. Valores entre `0.3` e `0.6` costumam ficar mais bonitos.
- `transparenciaDesligado` funciona do mesmo jeito para a porta desligada. Com `1`, ela some. Com `0.9`, sobra só um rastro bem fraco, para o jogador saber que ali tem uma porta.
- Mude os números com o jogo **parado**. O script lê as configurações só quando começa.

Etapa 6: efeitos de partículas e LED

Por quê: efeitos deixam a porta mais viva e avisam o jogador, de longe, se ela está ligada. O script já cuida deles: não precisa mexer no código, é só colocar os efeitos nas peças.

Partículas no laser

1. Clique com o botão direito no `Laser1`, vá em **Insert Object** e escolha **ParticleEmitter**.

2. Nas propriedades, ajuste: **Color** (cor das partículas), **Rate** (quantas por segundo, comece com 10), **Lifetime** (quanto tempo duram), **Speed** (velocidade), **Size** (tamanho, comece com 0.2) e **LightEmission** em `1`, para brilharem.
3. Para trocar o formato, mude a **Texture**. A Toolbox tem várias texturas de faísca e brilho.

As partículas dos lasers **ligam e desligam junto com o laser**.

LED no laser

- O jeito mais simples de LED é o material **Neon**: a peça brilha sozinha.
- Para iluminar em volta, insira um **PointLight** dentro do laser. **Color** é a cor da luz, **Brightness** a força e **Range** até onde ela chega. A luz também liga e desliga junto com o laser.
- Para iluminar só para um lado, use um **SpotLight** (em cone) ou um **SurfaceLight** (de uma face da peça).

LED e partículas na alavanca

- Insira um **PointLight** dentro da `Alavanca`. O script pinta essa luz de verde ou vermelho junto com a alavanca. Assim dá para ver de longe se a porta está ligada.
- Partículas dentro da alavanca também mudam de cor. Elas não desligam, porque a alavanca está sempre ali.
- Os efeitos que o script reconhece são: ParticleEmitter, PointLight, SpotLight, SurfaceLight, Beam, Trail, Fire, Smoke e Sparkles.

Se a porta estiver dentro de uma pasta de compra, os efeitos ficam escondidos até a compra, como qualquer outro item.

Etapa 7: textos flutuantes em cima dos botões

Por quê: até agora o jogador só descobria o que um botão vendia depois de comprar. Com um texto flutuando em cima de cada botão, ele vê o nome do item e o preço antes de decidir.

Como funciona: quando o jogo começa, o próprio script do botão cria um `BillboardGui`, uma placa de texto que flutua no ar e fica sempre virada para o jogador. Como a placa fica dentro do botão, ela some e aparece junto com ele: fica escondida até o botão ser liberado e some depois da compra.

Atenção: é preciso trocar o script de TODOS os botões de compra pelo script novo. O texto não é uma peça que se coloca à mão, ele faz parte do script. Um botão com o script antigo continua funcionando, mas fica sem texto.

Como fazer:

1. Pare o jogo, se ele estiver rodando. Mudanças feitas durante o Play são desfeitas no Stop.
2. Abra a peça `Botao` no Explorer. Dentro dela deve ficar **só o Script**. Se você tiver criado à mão um `BillboardGui` ou um `TextLabel` ali, pode apagar. O script novo apaga essas placas sozinho e cria a dele.
3. Dê dois cliques no script do botão, aperte **Ctrl+A**, **Delete** e cole o script novo do botão, entregue em aula.

4. Role até o fim do script no Studio e confira se a última linha é `-- FIM DO SCRIPT`. Se não for, o script foi colado pela metade: apague e cole de novo.
5. No topo do script, ajuste as configurações deste botão (veja a tabela mais abaixo).
6. Repita em todos os botões. O jeito mais rápido é acertar um botão, testar e depois duplicar a pasta dele (Ctrl+D), mudando só as configurações em cada cópia.

Todas as configurações do botão

São estas as linhas que podem ser mudadas em cada botão, todas no topo do script, na parte `CONFIGURAÇÕES DO BOTÃO`:

Variável	O que faz	Exemplo de valor
`preco`	Quanto custa a compra. Com `0`, o texto mostra "Grátis".	`150`
`nomeDoItem`	O nome que aparece na primeira linha do texto. Precisa ficar entre aspas.	`"Esteira"`
`ancorarAoComprar`	`true` deixa tudo preso no lugar depois da compra (decoração, paredes). `false` devolve cada peça como era (NPCs, portas, veículos).	`true` ou `false`
`alturaDoTexto`	Quantos studs o texto flutua acima do botão. Aumente se o texto encostar no botão.	`4`
`tamanhoDoTexto`	Tamanho do texto: `1` é o normal, `2` o dobro e `0.5` a metade.	`1.5`
`distanciaDoTexto`	A partir de quantos studs de distância o texto some, para não poluir a tela.	`80`
`corDoTexto`	A cor do texto, em vermelho, verde e azul (de 0 a 255).	`Color3.fromRGB(255, 215, 0)` (dourado)

Entendendo o que mudou no script:

```
-- cria o texto com o nome e o preço em cima do botão
local function criarTexto()
    local placa = Instance.new("BillboardGui")
    placa.Size = UDim2.new(0, 200 * tamanhoDoTexto, 0, 60 * tamanhoDoTexto)
    placa.StudsOffset = Vector3.new(0, alturaDoTexto, 0)
    placa.MaxDistance = distanciaDoTexto
    placa.Parent = botao

    -- (aqui o script cria o TextLabel com a cor e o contorno)

    if preco == 0 then
        texto.Text = nomeDoItem .. "\nGrátis"
    else
        texto.Text = nomeDoItem .. "\n$" .. preco
    end
end
```

- `criarTexto` cria a placa (`BillboardGui`) e o texto (`TextLabel`) dentro do botão. `StudsOffset` sobe a placa, `MaxDistance` esconde de longe e `TextStrokeTransparency = 0` coloca um contorno preto nas letras, para lerem bem em qualquer fundo.
- `\n` quebra a linha: o nome fica em cima e o preço embaixo.
- O texto é criado **antes** das anotações do botão. Assim ele é anotado junto com o botão e some e aparece com ele.
- A compra agora fica numa função própria, `tentarComprar`. O script termina com uma linha curta, que liga essa função ao toque no botão. O funcionamento da compra é o mesmo de antes:

```
-- liga a compra ao toque no botão
botao.Touched:Connect(tentarComprar)

-- FIM DO SCRIPT
```

Etapa 8: placa do dono

Por quê: com várias bases no mapa, os jogadores precisam saber de quem é cada uma. E, quando o dono morre (por exemplo, no laser de outra base), é chato renascer longe e ter que voltar andando. A placa resolve as duas coisas: mostra o nome do dono e é o ponto de renascimento dele.

Como fazer: a placa é uma peça nova, solta dentro da base, com um script próprio:

```
Tycoons
└── Tycoon1
    ├── BotaoDono
    ├── PlacaDono          (Part: a placa)
    │   └── PlacaDono      (Script)
    ├── Coletor
    ├── PainelArmazem
    └── Botao1 ...
```

1. Pare o jogo, se ele estiver rodando.
2. Clique com o botão direito na pasta `Tycoon1`, vá em **Insert Object** e escolha **Part**. Renomeie para `PlacaDono`.
3. Coloque **Size** em `8, 4, 0.5` e marque **Anchored**. Deixe a placa em pé, num lugar visível, com espaço livre na frente: é ali que o dono vai renascer.
4. Clique com o botão direito na `PlacaDono`, insira um **Script**, renomeie para `PlacaDono`, apague o que vier escrito e cole o script da placa entregue em aula.
5. Role até o fim do script e confira se a última linha é `-- FIM DO SCRIPT`.
6. Dê **Play** e pegue a base no `BotaoDono`. A placa deve trocar de "Base livre" para "Base de" e o seu nome.
7. Se o texto aparecer do lado errado da placa, pare o jogo, selecione a placa e aperte **Ctrl+R** duas vezes para girá-la, ou mude a configuração `ladoDoTexto`.
8. Copie a `PlacaDono` para dentro das outras bases. O script descobre sozinho de qual base é cada placa.

A `PlacaDono` precisa ficar **solta dentro da base**, e não dentro de uma pasta de botão, para aparecer sempre. Qualquer placa de texto feita à mão dentro dela é apagada quando o jogo começa: o script cria a dele com as configurações.

Configurações do texto

Todas ficam no topo do script, na parte `CONFIGURAÇÕES DO TEXTO`:

Variável	O que faz	Exemplo de valor
`textoLivre`	Texto quando a base não tem dono	`"Base livre"`
`textoDono`	Texto que vem antes do nome do dono	`"Dono:"`
`quebrarLinha`	`true` põe o nome na linha de baixo; `false` deixa tudo na mesma linha	`true` ou `false`
`usarApelido`	`true` mostra o apelido do jogador (DisplayName); `false` mostra o nome de usuário	`true` ou `false`
`tamanhoDoTexto`	`0` faz o texto ocupar a placa toda; um número fixa o tamanho das letras	`0`, `30`, `50`
`fonte`	Fonte das letras	`Enum.Font.Arcade`
`corDoTexto`	Cor das letras	`Color3.fromRGB(255, 215, 0)`
`mostrarContorno`	`true` coloca um contorno em volta das letras	`true` ou `false`
`corDoContorno`	Cor do contorno	`Color3.fromRGB(0, 0, 0)`
`corDoFundo`	Cor do fundo atrás do texto	`Color3.fromRGB(30, 30, 30)`
`transparenciaDoFundo`	`0` é fundo sólido, `1` é sem fundo	`0`, `0.5`, `1`
`ladoDoTexto`	Lado da placa onde fica o texto	`Enum.NormalId.Front` ou `Back`

Configurações do renascimento

Variável	O que faz	Exemplo de valor
`renascerNaPlaca`	`true` faz o dono renascer na frente da placa; `false` desliga	`true` ou `false`
`distanciaRenascer`	Quantos studs longe da placa o dono renasce	`5`
`alturaRenascer`	Altura do renascimento. Aumente se o personagem nascer preso no chão	`3`

Entendendo as partes principais do script:

O texto muda sozinho quando o dono muda

```
-- atualiza o texto e o renascimento quando o dono muda
tycoon:GetAttributeChangedSignal("Dono"):Connect(atualizarDono)
```

- A placa usa a mesma etiqueta `Dono` que o `BotaoDono` grava na base. Quando a etiqueta muda (alguém pega a base ou o dono sai), a função `atualizarDono` troca o texto.
- `Players:GetPlayerByUserId` transforma o número guardado na etiqueta no jogador de verdade, para o script poder pegar o nome dele.

O renascimento

```
conexaoRenascer = dono.CharacterAdded:Connect(levarParaPlaca)
```

- `CharacterAdded` dispara toda vez que o personagem do jogador é criado, inclusive depois de morrer. Toda vez que isso acontece com o dono, a função `levarParaPlaca` roda.

```
-- leva o personagem para a frente do lado do texto
personagem:PivotTo(CFrame.lookAt(posicao, posicao + olhar))
```

- `PivotTo` move o personagem inteiro de uma vez para a frente da placa, já virado para o lado oposto a ela.
- O dono sempre renasce na frente do **mesmo lado do texto**. Se `ladoDoTexto` mudar para `Back`, o renascimento também vai para trás da placa.
- Quando a base troca de dono, a ligação com o dono antigo é desfeita (`Disconnect`). Assim, só o dono atual renasce na placa.

Tamanho do texto

```
if tamanhoDoTexto == 0 then
    texto.TextScaled = true
else
    texto.TextScaled = false
    texto.TextSize = tamanhoDoTexto
end
```

- Com `0`, o `TextScaled` deixa o texto do maior tamanho que cabe na placa. Com outro número, as letras ficam com aquele tamanho fixo. Numa placa `8, 4`, um tamanho de `50` fica bom.

Testar e problemas comuns

Na aba **Test**, escolha **2 Players**, clique em **Start** e confira:

- Ao chegar perto da alavanca, aparece "E Ligar laser"
- Ao apertar E, a alavanca fica verde e o laser aparece
- O dono atravessa o laser ligado sem morrer
- O outro jogador não consegue usar a alavanca
- O outro jogador morre ao atravessar o laser ligado
- Com a alavanca vermelha, o laser some e ninguém morre
- As partículas e luzes do laser apagam junto com ele
- Cada botão mostra o nome do item e o preço, e o texto some depois da compra
- A placa do dono mostra "Base livre" e troca para o nome de quem pegou a base

- Ao morrer, o dono renasce na frente da placa da base dele

Problema	Causa provável	Solução
O E não aparece	O jogador está longe ou a porta ainda não foi comprada	Chegue mais perto, aumente `distanciaInteracao` ou compre a porta
Erro "Infinite yield" falando de Alavanca	A peça não se chama `Alavanca` ou não está direto na pasta	Renomeie a peça e coloque-a direto na pasta `PortaLaser`
A alavanca virou laser e mata quem chega perto	A alavanca está com outro nome	Renomeie para `Alavanca`, exatamente assim
Todo mundo morre, até o dono	A pasta `PortaLaser` está fora de uma base	Mova a pasta para dentro de uma base em `Tycoons`
O jogador parado no laser não morre	O `Touched` só dispara quando o personagem se mexe	É normal: ao dar um passo, ele morre
O E aparece na alavanca escondida depois do reset	O `ProximityPrompt` não foi criado no Studio	Crie o `ProximityPrompt` dentro da alavanca no Studio
O texto do botão não aparece	O botão ainda está com o script antigo, a base não tem dono ou o jogador está longe	Troque o script, pegue a base no `BotaoDono` e chegue perto (ou aumente `distanciaDoTexto`)
Erro "Expected ')' ... got <eof>"	O script foi colado pela metade e o final ficou de fora	Apague e cole de novo; a última linha precisa ser `-- FIM DO SCRIPT`
Erro "attempt to index nil with 'Text'"	O botão está com uma versão antiga do script e uma placa feita à mão	Troque pelo script novo, que apaga as placas feitas à mão
A placa mostra o texto do lado errado, ou o dono renasce atrás dela	A frente da placa está virada para o outro lado	Gire a placa com Ctrl+R ou mude `ladoDoTexto`
O dono renasce preso no chão ou dentro de algo	Não há espaço livre na frente da placa, ou a altura é baixa	Deixe espaço livre e aumente `alturaRenascer` ou `distanciaRenascer`
O texto da placa sai cortado	`tamanhoDoTexto` grande demais para a placa	Use `0` (automático), diminua o número ou aumente a placa
O botão parou de comprar	O script deu erro e parou antes da parte da compra	Abra o Output (View → Output), veja a linha vermelha e confira as linhas acima desta tabela
As partículas não apagam com o laser	O efeito está fora das peças da pasta	Coloque o efeito dentro de uma peça laser

Próximas aulas

- **Itens da Toolbox à venda**: poderes e armas que o jogador pode comprar na base.

Tarefa em aula

- **Faça toda a implementação do arquivo**
- **Coloque pelo menos uma porta laser em cada base**
- **Coloque nome e preço em todos os botões de compra**
- **Coloque uma placa do dono em cada base**
- **Desafio extra: faça uma porta com vários lasers formando uma grade**